

File Systems

Note: This topic isn't covered in your source notes — it's drafted from general SDE-interview OS knowledge, in the same style as the other topics. Flag anything you want trimmed, expanded, or corrected.

1. Why File Systems Exist

Secondary storage (disk/SSD) is just a huge, flat array of addressable blocks — the hardware itself has no concept of "files" or "folders." The **file system** is the OS layer that imposes structure on top of that raw block array: naming, organization, access control, and a way to find and grow/shrink data without the user ever touching raw block numbers.

One-line definition: A file system is the OS component that manages how data is named, stored, organized, and retrieved on persistent storage.

2. The File Abstraction

A **file** is a named collection of related information, persistently stored.

Common attributes (metadata) every file carries

- Name, type/extension
- Size
- Location (which blocks on disk hold its data)
- Owner, permissions (read/write/execute, per user/group/others)
- Timestamps — created, last modified, last accessed
- A unique identifier (e.g., an **inode number** on Unix-like systems)

Common operations

- Create, delete, open, close, read, write, append, seek (reposition the read/write pointer), truncate, rename.

Access methods

- **Sequential access** — read bytes/records in order, from the beginning (e.g., streaming a log file). Matches how magnetic tape historically worked.

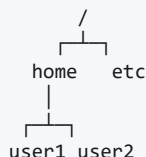
- **Direct (random) access** — jump straight to any block/offset without reading everything before it (e.g., a database file, `seek()` + `read()`).
- **Indexed access** — maintain an index mapping logical record numbers to physical block locations, enabling fast lookups by key rather than by position (conceptually layered on top of direct access).

3. Directory Structures

A **directory** is itself just a special file that maps names to file locations/metadata (essentially a table of `{filename → inode/file-control-block}`).

Structure	Idea	Limitation
Single-level	One directory for the entire system	No two files can share a name, at all — completely impractical beyond a toy system
Two-level	One directory per user, under one master directory	Users are isolated from each other, but still can't organize their <i>own</i> files into sub-groups
Tree-structured	Directories can contain subdirectories, forming a hierarchy (what virtually every real OS uses)	A file can only have one path/parent — no legitimate way to make a file appear in two places
Acyclic-graph	Like a tree, but allows shared subdirectories/files via links (hard links / symbolic links) — the same file can appear under multiple directories	Requires careful handling of deletion (see hard vs. soft links below) — no <i>cycles</i> allowed
General graph	Like acyclic-graph, but cycles are allowed	Needs cycle detection to prevent infinite loops during traversal (e.g., <code>du</code> , backup tools) and to correctly garbage-collect files with circular references

Tree-structured (most common):



Interview soundbite: "Real-world file systems use a tree structure for simplicity, extended with an acyclic-graph capability via links so the same file can be shared across multiple locations without duplicating it — but true cycles are avoided because they'd break simple recursive traversal and reference counting."

Hard Links vs Symbolic (Soft) Links

	Hard Link	Symbolic (Soft) Link
What it is	Another directory entry pointing to the same inode (same underlying data)	A separate small file that just stores the path to the target
Survives target deletion?	Yes — data persists as long as any hard link (or the original) still references that inode	No — becomes a "dangling link" if the target is deleted
Can span file systems/partitions?	No — must be on the same file system (same inode table)	Yes — just a stored path string, works across file systems
Can link to a directory?	Generally disallowed (to prevent cycles in the directory tree)	Yes, freely
Reference counting	Inode keeps a link count ; data is only actually freed when the link count hits 0	Not counted — the target file's link count is unaffected

Interview soundbite: "A hard link is another name for the exact same inode — deleting the original doesn't remove the data as long as another hard link exists. A symbolic link is just a pointer file storing a path — delete the target, and the symlink breaks."

4. File Allocation Methods (how a file's blocks are laid out on disk)

4.1 Contiguous Allocation

Each file occupies a **single contiguous run** of blocks.

File A: blocks [10-14] File B: blocks [15-17]

- ✓ Fast sequential *and* direct access (block `i` of the file = `start_block + i`, a simple calculation).
- ✗ Same fragmentation problems as contiguous *memory* allocation (external fragmentation as files are created/deleted/resized); hard to grow a file in place if the next block is already taken by another file.

4.2 Linked Allocation

Each file is a **linked list of blocks**, scattered anywhere on disk; each block stores a pointer to the next block of the same file.

```
File A: [Block 9 | data | →12] → [Block 12 | data | →17] → [Block 17 | data | →NULL]
```

-  No external fragmentation (any free block works); files can grow easily (just link a new block wherever there's space).
-  **No efficient direct access** — to reach block i , you must traverse i pointers from the start, sequentially. Also, a single corrupted pointer breaks the chain for everything after it, and each block wastes a little space storing the "next" pointer.
- **FAT (File Allocation Table)** variant: instead of storing the "next" pointer *inside* each data block, keep the entire linked structure in one dedicated table (the FAT) held in memory — this at least avoids extra disk seeks purely to read "next" pointers, and is exactly what the FAT/FAT32 file system is named after.

4.3 Indexed Allocation

Each file has a dedicated **index block** containing pointers to all of that file's data blocks.

```
File A's index block: [→ Block 4, → Block 19, → Block 2, → Block 30]
```

-  Supports true direct access (look up entry i in the index — no traversal needed) *and* no external fragmentation.
-  The index block itself uses space, and for very large files, a single index block may not have room for enough pointers.
- **Fix for large files — multi-level / hybrid indexing** (this is exactly how Unix **inodes** work):
- A fixed number of **direct pointers** (straight to data blocks) for small files.
- A **single indirect pointer** (points to a block that itself is full of pointers to data blocks) for medium files.
- A **double indirect pointer** (points to a block of pointers to blocks of pointers to data) for large files.
- A **triple indirect pointer**, for extremely large files.

```
inode
├── direct[0..11]    → data blocks directly
├── single indirect  → [pointer block] → data blocks
├── double indirect  → [pointer block] → [pointer blocks] → data blocks
└── triple indirect  → ... one level deeper still
```

Why this layered design: small files (the overwhelming majority on most systems) are accessed with **zero extra indirection** (direct pointers), keeping common-case access fast, while the rare very-large file can still be represented without needing an impractically large flat index — the indirection cost is only paid when it's actually needed.

Comparison

	Contiguous	Linked	Indexed
Direct access	✓ Fast	✗ Slow (sequential traversal)	✓ Fast
External fragmentation	Yes	No	No
Growing a file in place	Hard	Easy	Easy
Overhead	None extra	Per-block pointer	Index block(s)
Real-world example	Rare in modern general-purpose FS	FAT (via table-based variant)	ext-family, NTFS (via inode/MFT-like structures)

5. Free Space Management

The file system must track which blocks are **currently unused**, so it knows where new file data can go.

- **Bitmap (bit vector)** — one bit per block: `1` = allocated, `0` = free. Compact, and fast to scan for a run of free blocks (useful for allocating contiguous or near-contiguous space), but the bitmap itself must be scanned linearly to find free bits.
- **Linked list of free blocks** — each free block points to the next free block. No wasted space computing/storing a separate bitmap, but finding a contiguous *run* of free blocks requires walking the list — slow.
- **Grouping** — the first free block stores addresses of the next N free blocks (one of which, in turn, stores the next N after that) — speeds up finding multiple free blocks at once compared to a plain linked list.
- **Counting** — since blocks tend to be freed/allocated in contiguous clusters in practice, store `(start block, count of contiguous free blocks)` pairs instead of tracking every free block individually — much more compact when free space is fragmented into a manageable number of runs rather than being scattered block-by-block.

6. Directory Implementation

- **Linear list** — a simple array/list of `{filename, metadata/inode pointer}` entries. Simple, but $O(n)$ to find a file by name (linear search) — noticeably slow for directories with many entries.
- **Hash table** — hash the filename to quickly locate its entry, giving close to $O(1)$ average lookup. Adds complexity for handling hash collisions and requires rehashing/resizing as the directory grows.

7. Journaling — durability against crashes

The problem: a multi-step file system operation (e.g., "allocate a block, update the file's inode, update the free-space bitmap") can be **interrupted midway** by a crash or power loss, leaving the file system in an inconsistent state (e.g., a block marked both "in use by file A" and "free").

The fix — a journal (write-ahead log): before actually making changes to the real file-system structures, the OS first writes a description of the intended changes to a separate, append-only **journal**. If a crash happens mid-operation, on reboot the OS replays the journal to either **complete** or **cleanly roll back** the interrupted operation — guaranteeing the file system structure itself is never left in a torn, half-updated state, even if some data written by the interrupted operation is lost.

Interview soundbite: "Journaling doesn't prevent data loss for whatever was being written at crash time — it guarantees the *file system's own metadata structures* stay internally consistent, by writing an intent log before making the real changes, so a crash mid-operation can always be cleanly completed or rolled back on reboot."

- **ext3/ext4** (Linux) and **NTFS** (Windows) are both journaling file systems.
- Older **FAT32** is *not* journaled — this is exactly why an interrupted write on a FAT32 USB drive is far more likely to leave it corrupted than an equivalent interruption on ext4/NTFS.

8. Mounting

A raw storage device isn't usable until its file system is **mounted** — attached to a specific point (the **mount point**) in the OS's existing directory tree, so that path lookups can transparently cross from one file system onto another.

Example (Linux): plugging in a USB drive and it appearing at `/media/usb` — everything under `/media/usb` is actually served by the USB drive's own file system, but from the user's perspective it's just another subtree of the same overall directory hierarchy.

Interview Questions With Answers

Q1. What is the core job of a file system, given that disks are just flat arrays of blocks?

Answer: The file system imposes structure — naming, hierarchical organization (directories), access control, and metadata (size, timestamps, permissions) — on top of what hardware sees only as a flat

array of addressable blocks. It also manages how a file's data is actually laid out across those blocks and tracks which blocks are free.

Q2. What's the difference between sequential, direct, and indexed file access?

Answer: Sequential access reads a file in order from the beginning, like a tape. Direct (random) access can jump to any position immediately without reading everything before it. Indexed access maintains a separate index mapping logical keys/record numbers to physical locations, enabling fast lookups by key — conceptually built on top of direct access.

Q3. Compare tree-structured, acyclic-graph, and general-graph directory structures.

Answer: A tree allows arbitrary nesting of directories but each file/directory has exactly one parent/path. An acyclic-graph structure additionally allows a file or directory to be shared and appear under multiple parents via links, without forming loops. A general graph goes further and permits cycles, but then requires cycle detection to avoid infinite traversal loops (e.g., when recursively listing files) and to correctly determine when a file with circular references can actually be deleted.

Q4. Explain the difference between a hard link and a symbolic link, including what happens when the original file is deleted.

Answer: A hard link is a second directory entry pointing to the exact same inode (the same underlying data and metadata) as the original — the inode tracks a link count, and the actual data is only freed once that count reaches zero, so deleting "the original" name still leaves the data intact as long as another hard link exists. A symbolic link is a separate small file that just stores the *path* to the target; it doesn't affect the target's link count at all, so deleting the target leaves the symlink "dangling" — pointing to something that no longer exists.

Q5. Why can't linked allocation support efficient direct/random access?

Answer: Because each block only stores a pointer to the *next* block in the file — to reach the *i*-th block, the OS must start at the first block and follow *i* pointers sequentially; there's no way to jump straight to an arbitrary block's location without traversing everything before it.

Q6. How does indexed allocation solve the direct-access problem that linked allocation has?

Answer: Indexed allocation keeps a dedicated index block containing direct pointers to every one of the file's data blocks. To access block *i*, the OS simply reads entry *i* from the index block — an $O(1)$ lookup — rather than traversing a chain.

Q7. Explain the multi-level (direct / single-indirect / double-indirect) indexing scheme used by inodes, and why it's designed this way.

Answer: An inode has a small fixed number of direct pointers straight to data blocks (fast, zero extra indirection, sufficient for the vast majority of small files), a single-indirect pointer to a block full of

further pointers (handling medium-sized files), and double/triple-indirect pointers that add additional layers of pointer-blocks for very large files. This design keeps the common case (small files) fast with no extra indirection overhead, while still being able to represent arbitrarily large files without needing one impractically huge flat index for every file regardless of size.

Q8. What problem does journaling solve, and what exactly does it guarantee (and not guarantee)?

Answer: It solves the problem of a multi-step file system operation being interrupted mid-way by a crash or power loss, which could otherwise leave core file system structures (like the free-space bitmap or an inode) in an inconsistent state. Journaling guarantees that, by logging intended changes before applying them, an interrupted operation can always be cleanly completed or rolled back on reboot — the file system's own structural integrity is preserved. It does **not** guarantee that the specific data being written at the moment of the crash is preserved — only that the file system as a whole doesn't end up structurally corrupted.

Q9. Why is a FAT32 USB drive more prone to corruption from an unsafe removal than an ext4 or NTFS drive?

Answer: FAT32 is not a journaling file system — it doesn't log intended metadata changes before applying them. If the drive is removed mid-write, a multi-step update (e.g., updating the file allocation table and a directory entry) can be interrupted between steps, leaving the file system's internal structures inconsistent with no recovery mechanism. ext4 and NTFS are journaling file systems, so an interrupted operation can be detected and either completed or rolled back cleanly on the next mount.

Q10. Compare bitmap, linked-list, and counting approaches to free space management.

Answer: A bitmap uses one bit per block (allocated/free) — compact and fast to scan for contiguous runs, but the whole bitmap must be searched to find free space. A linked list of free blocks avoids a separate bitmap structure but is slow to find a *contiguous run* of free blocks, since you must walk the list. Counting stores compact `(start, length)` pairs for contiguous free runs, which is much more space-efficient than tracking every free block individually, especially when free space naturally clusters into a manageable number of contiguous stretches (which it usually does, since blocks tend to be freed together when a file is deleted).

Q11. What does "mounting" a file system actually mean?

Answer: Mounting attaches a storage device's file system to a specific point (the mount point) within the OS's existing directory hierarchy, so that path lookups can seamlessly cross from the main file system onto the mounted one. Without mounting, the OS would have no way to expose a separate device's files as part of the same navigable directory tree the user already interacts with.

Q12. Scenario: You create a hard link and a symbolic link to the same file, then delete the original file. What happens if you try to access the file through each link?

Answer: Through the **hard link**, the file is still fully accessible — a hard link is just another name for the same inode, and since the inode's link count only dropped from (at least) 2 to 1 rather than to 0, the underlying data is never freed; the OS doesn't even distinguish which name was "the original." Through the **symbolic link**, the access fails — the symlink only stores a path string pointing at the original filename, which no longer resolves to anything now that it's been deleted, resulting in a "file not found" / broken-link error.